

Национальный исследовательский университет „Высшая школа экономики“
Программная инженерия

Разработка консольной игры Gomoku

Демьянова Таисия Владимировна

Москва, 2025

Оглавление

1. Постановка задачи	1
2. Описание игры.....	2
2.1 Основной функционал	2
2.2 Интерфейс.....	3
3. Программная реализация.....	5
4. Список использованных источников	7
Приложение 1 (листинг)	8
Листинг 1. Program.cs — точка входа приложения	8
Листинг 2. Player.cs — сущность игрока.....	8
Листинг 3. ConsoleHelper.cs — вывод текста с цветовым оформлением	8
Листинг 4. Board.cs — игровое поле и проверка победы.....	9
Листинг 5. Game.cs — основной игровой процесс.....	9
Листинг 6. Menu.cs — главное меню.....	11
Листинг 7. SaveManager.cs — сохранение и загрузка данных.....	12
Листинг 8. TwoDimensionalIntArrayConverter.cs — сериализация массива int[,].....	13
Листинг 9. Gomoku.csproj — файл проекта.....	13
Ссылка на полный исходный код проекта.....	13

1. Постановка задачи

Кратко: «Реализовать консольную игру Gomoku на .NET, поле 15×15 , два игрока, победа — 5 подряд по горизонтали/вертикали/диагонали».

В рамках учебного проекта преподавателем была поставлена задача разработать консольную игру «Gomoku» (крестики-нолики на поле 15×15).

Основные требования к реализации:

- создать игровое поле размером 15×15 клеток;
- реализовать возможность игры двух участников;
- предусмотреть проверку корректности хода;
- обеспечить определение победителя при последовательности из пяти символов по горизонтали, вертикали или диагонали;
- разработать консольный интерфейс с цветовым выделением символов игроков;
- реализовать меню для запуска новой игры, продолжения, сохранения и загрузки состояния, а также отображения таблицы рекордов.

2. Описание игры

2.1 Основной функционал

- Игровое поле 15×15.
- Два игрока (имя, цвет, счёт).
- Ходы по координатам.
- Проверка корректности хода.
- Проверка победы (5 подряд).
- Учёт очков и количества ходов.
- Возможность выхода в меню

Разработанная консольная игра Gomoku реализует классическую механику крестиков-ноликов на поле 15×15. Игровой процесс предусматривает участие двух игроков, каждый из которых имеет имя, цвет и счёт. Игроки поочерёдно вводят координаты хода, система проверяет корректность ввода и определяет победителя при последовательности из пяти символов. Реализован учёт количества ходов, переключение игроков и возможность выхода в меню.

2.2 Интерфейс

```
-----  
Player1          Red          0  
Player2          Blue         0  
-----  
  
=== Gomoku Menu ===  
1. Start New Game  
2. Continue Game  
3. Load Last Save  
4. Save Current Game  
5. Settings  
6. Top Scores  
7. Exit  
Enter your choice: █
```

Рисунок 1 – интерфейс меню

```
Name          Color          Score  
-----  
Gomoku Game  
-----  
    00 01 02 03 04 05 06 07 08 09 10 11 12 13 14  
00 . . . . . . . . . . . . . . .  
01 . . . . . . . . . . . . . . .  
02 . . . . . . . . . . . . . . .  
03 . . . . . . . . . . . . . . .  
04 . . . . . . . . . . . . . . .  
05 . . . . . . . . . . . . . . .  
06 . . . . . . . . . . . . . . .  
07 . . . . . . . . . . . . . . .  
08 . . . . . . . . . . . . . . .  
09 . . . . . . . . . . . . . . .  
10 . . . . . . . . . . . . . . .  
11 . . . . . . . . . . . . . . .  
12 . . . . . . . . . . . . . . .  
13 . . . . . . . . . . . . . . .  
14 . . . . . . . . . . . . . . .  
Player1's turn (Symbol: X)  
Enter row and column (e.g., 3 4) or "menu": █
```

Рисунок 2 – интерфейс игрового поля

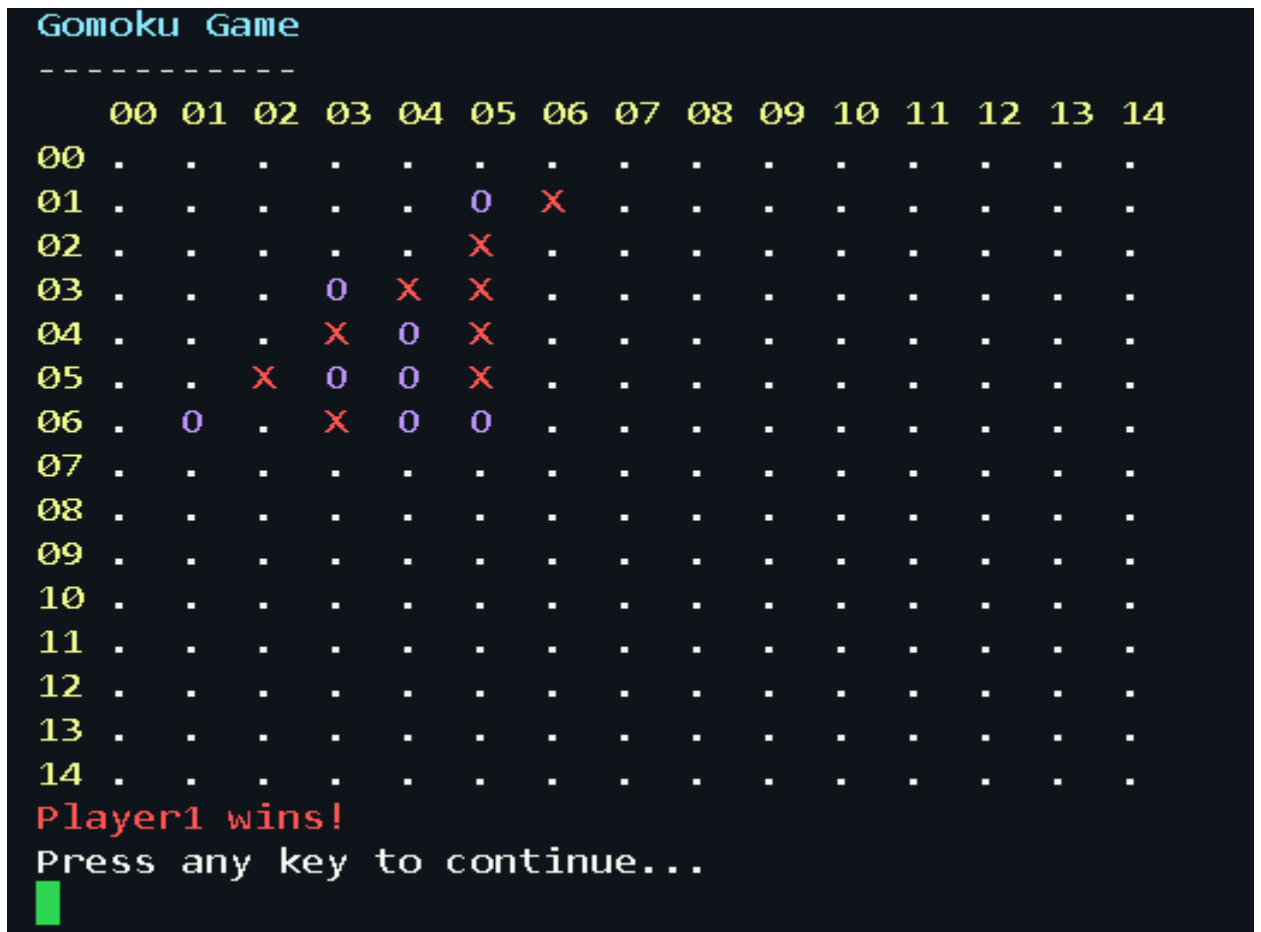


Рисунок 3 – интерфейс игрового процесса

- Консольный вывод поля с координатами.
- Цветовое выделение символов (X — **красный**, O — **синий**).
- Сообщения о ходе, победе, ошибках.
- Главное меню с пунктами: новая игра, продолжение, сохранение, загрузка, настройки, рекорды, выход.

Интерфейс игры построен на консольном выводе. Игровое поле отображается с координатами строк и столбцов, символы игроков выделяются цветом (X — **красный**, O — **синий**). В процессе игры выводятся сообщения о текущем ходе, победе или ошибках ввода. Главное меню предоставляет доступ к запуску новой игры, продолжению текущей, сохранению и загрузке состояния, настройкам игроков и таблице рекордов.

3. Программная реализация

Программная реализация выполнена на языке C# в среде .NET 9.0. Проект представляет собой консольное приложение, состоящее из следующих классов:

- Board — отвечает за игровое поле, выполнение ходов и проверку условий победы.
- ConsoleHelper — обеспечивает цветной вывод текста и оформление интерфейса.
- Game — управляет игровым процессом, циклами ходов, переключением игроков и проверкой победы.
- Menu — реализует главное меню, включая запуск новой игры, сохранение, загрузку, настройки и таблицу рекордов.
- Player — описывает сущность игрока (имя, цвет, счёт).
- SaveManager — отвечает за сохранение и загрузку состояния игры и таблицы рекордов в формате JSON.
- TwoDimensionalIntArrayConverter — обеспечивает сериализацию и десериализацию двумерного массива `int[,]` для корректного сохранения игрового поля.
- Program — является точкой входа и запускает меню.
- Файл проекта (.csproj) задаёт параметры сборки и указывает целевую платформу .NET 9.0.

На рисунке 4 представлена блок-схема программного обеспечения, отражающая взаимодействие основных компонентов.

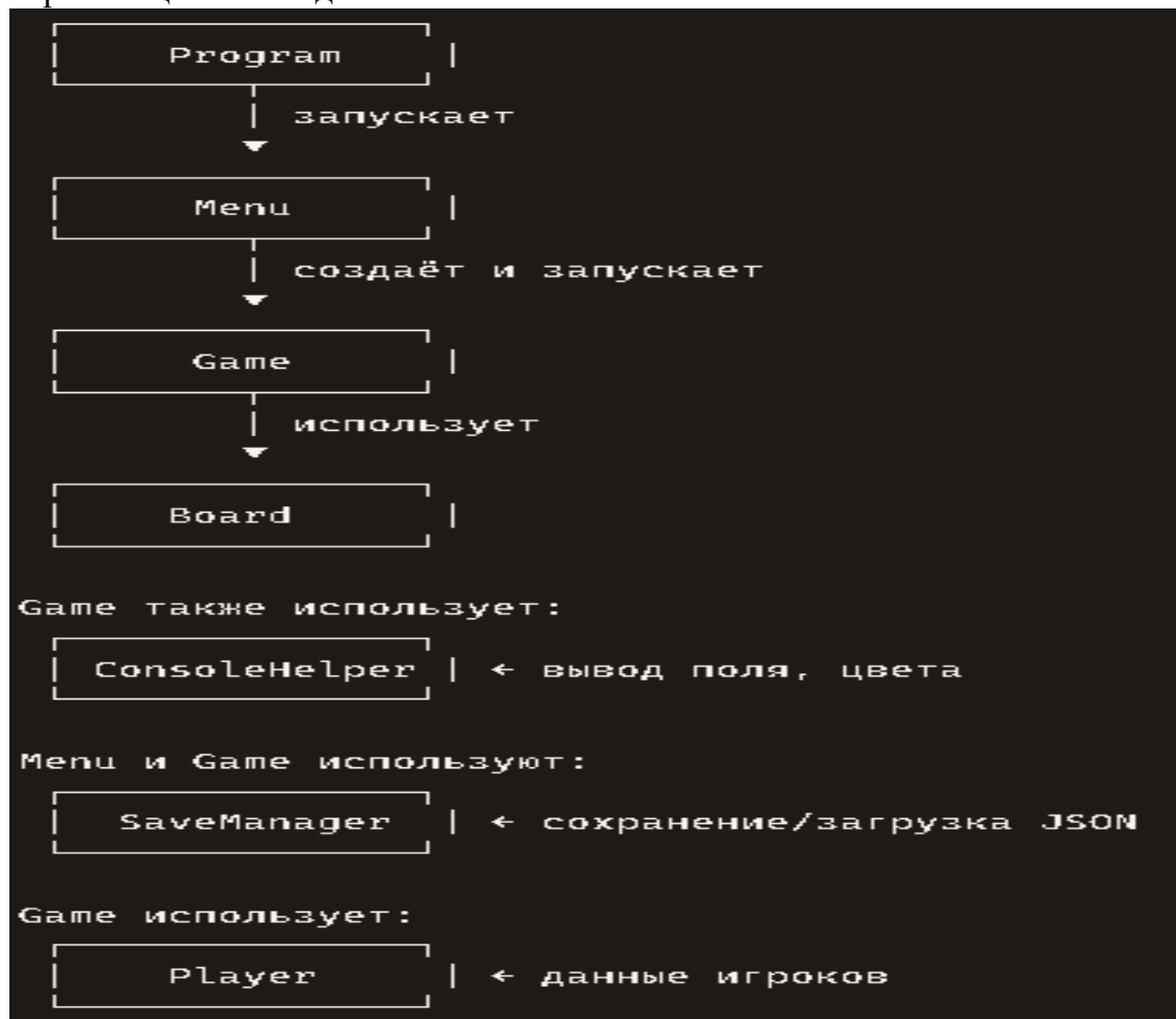


Рисунок 4 - блок-схема программного обеспечения

4. Список использованных источников

1. Microsoft Docs. Документация по языку C# и платформе .NET 9.0.
URL: <https://learn.microsoft.com/dotnet/csharp/>
2. Microsoft Docs. Пространство имён System.Text.Json.
URL: <https://learn.microsoft.com/dotnet/api/system.text.json>
3. Правила настольной игры Gomoku (Пять в ряд).
URL: <https://en.wikipedia.org/wiki/Gomoku>
4. Дополнительные ресурсы по работе с консольным интерфейсом в C#.
URL: <https://learn.microsoft.com/dotnet/api/system.console>

Приложение 1 (листинг)

В данном приложении приведены основные фрагменты исходного кода консольной игры **Gomoku**, реализованной на платформе .NET 9.0. Код оформлен моноширинным шрифтом (Consolas 10 pt). Полный проект доступен в репозитории GitHub (см. ссылку в конце приложения).

Листинг 1. Program.cs — точка входа приложения

```
using Gomoku;

new Menu().Display();
```

Листинг 2. Player.cs — сущность игрока

```
namespace Gomoku;

public class Player
{
    public string Name { get; set; } = null!;
    public ConsoleColor Color { get; set; }
    public int Score { get; set; }
}
```

Листинг 3. ConsoleHelper.cs — вывод текста с цветовым оформлением

```
namespace Gomoku;

public static class ConsoleHelper
{
    public static void WriteColored(string msg, ConsoleColor color)
    {
        var old = Console.ForegroundColor;
        Console.ForegroundColor = color;
        Console.Write(msg);
        Console.ForegroundColor = old;
    }

    public static void WriteLineColored(string msg, ConsoleColor color)
    {
        WriteColored(msg + "\n", color);
    }

    public static void ClearWithHeader(string header)
    {
        Console.Clear();
        WriteLineColored(header, ConsoleColor.Cyan);
    }
}
```

Листинг 4. Board.cs — игровое поле и проверка победы

```
namespace Gomoku;

public class Board
{
    private const int Size = 15;
    public int[,] Cells { get; }

    public Board()
    {
        Cells = new int[Size, Size];
    }

    public bool MakeMove(int x, int y, int player)
    {
        if (x < 0 || y < 0 || x >= Size || y >= Size || Cells[x, y] != 0)
            return false;

        Cells[x, y] = player;
        return true;
    }

    public bool CheckWin(int player)
    {
        for (int i = 0; i < Size; i++)
            for (int j = 0; j < Size; j++)
                if (Cells[i, j] == player &&
                    (Check(i, j, 1, 0, player) ||
                     Check(i, j, 0, 1, player) ||
                     Check(i, j, 1, 1, player) ||
                     Check(i, j, 1, -1, player)))
                    return true;

        return false;
    }

    private bool Check(int x, int y, int dx, int dy, int p)
    {
        for (int k = 0; k < 5; k++)
        {
            int nx = x + dx * k, ny = y + dy * k;
            if (nx < 0 || ny < 0 || nx >= Size || ny >= Size || Cells[nx, ny]
                != p)
                return false;
        }
        return true;
    }
}
```

Листинг 5. Game.cs — основной игровой процесс

```
namespace Gomoku;

public class Game
{
    public Player Player1 { get; }
    public Player Player2 { get; }
    public Board Board { get; private set; }
    public int CurrentPlayer { get; private set; }
    public bool IsStarted { get; private set; }

    public Game()
    {

```

```

        Player1 = new Player { Name = "Player1", Color = ConsoleColor.Red };
        Player2 = new Player { Name = "Player2", Color = ConsoleColor.Blue };
        InitializeNewGame();
    }

    public void InitializeNewGame()
    {
        Board = new Board();
        CurrentPlayer = 1;
        IsStarted = false;
    }

    public void Play()
    {
        IsStarted = true;

        while (true)
        {
            ConsoleHelper.ClearWithHeader("Gomoku Game");

            Console.Write($"{GetCurrent().Name}'s move: ");
            string input = Console.ReadLine() ?? "";

            if (input == "menu")
                break;

            if (TryParse(input, out int x, out int y) &&
                Board.MakeMove(x, y, CurrentPlayer))
            {
                if (Board.CheckWin(CurrentPlayer))
                {
                    ConsoleHelper.WriteLineColored($"{GetCurrent().Name}
wins!", GetCurrent().Color);
                    GetCurrent().Score++;
                    break;
                }
                SwitchPlayer();
            }
            else
            {
                ConsoleHelper.WriteLineColored("Invalid move.",
ConsoleColor.Red);
            }
        }
    }

    private Player GetCurrent() => CurrentPlayer == 1 ? Player1 : Player2;
    private void SwitchPlayer() => CurrentPlayer = CurrentPlayer == 1 ? 2 :
1;

    private bool TryParse(string s, out int x, out int y)
    {
        x = y = -1;
        var p = s.Split(' ');
        return p.Length == 2 && int.TryParse(p[0], out x) &&
int.TryParse(p[1], out y);
    }
}

```

Листинг 6. Menu.cs — главное меню

```
namespace Gomoku;

public class Menu
{
    private Game _game = new Game();

    public void Display()
    {
        int choice = 0;

        while (choice != 7)
        {
            ConsoleHelper.ClearWithHeader("Gomoku Menu");
            Console.WriteLine("1. Start New Game");
            Console.WriteLine("2. Continue Game");
            Console.WriteLine("3. Load Last Save");
            Console.WriteLine("4. Save Current Game");
            Console.WriteLine("5. Settings");
            Console.WriteLine("6. Top Scores");
            Console.WriteLine("7. Exit");

            if (!int.TryParse(Console.ReadLine(), out choice))
                continue;

            switch (choice)
            {
                case 1: _game.InitializeNewGame(); _game.Play(); break;
                case 2: if (_game.IsStarted) _game.Play(); break;
                case 3: _game = SaveManager.LoadGame() ?? _game; break;
                case 4: if (_game.IsStarted) SaveManager.SaveGame(_game);
                    break;
                case 5: Settings(); break;
                case 6: TopScores(); break;
            }
        }

        private void Settings()
        {
            ConsoleHelper.ClearWithHeader("Settings");
            Console.Write("Enter new name for Player 1: ");
            string name = Console.ReadLine() ?? "Player1";
            _game.ChangePlayer(1, name, ConsoleColor.Red);
        }

        private void TopScores()
        {
            var scores = SaveManager.LoadScores();
            ConsoleHelper.ClearWithHeader("Top Scores");

            foreach (var s in scores)
                Console.WriteLine($"{s.Name}: {s.Score}");

            Console.ReadKey(true);
        }
    }
}
```

Листинг 7. SaveManager.cs — сохранение и загрузка данных

```
using System.Text.Json;

namespace Gomoku;

public static class SaveManager
{
    private const string GameFile = "game_save.json";
    private const string ScoreFile = "score_save.json";

    public static void SaveGame(Game game)
    {
        string json = JsonSerializer.Serialize(
            game,
            new JsonSerializerOptions
            {
                WriteIndented = true,
                Converters = { new TwoDimensionalIntArrayConverter() }
            });

        File.WriteAllText(GameFile, json);
    }

    public static Game? LoadGame()
    {
        if (!File.Exists(GameFile))
            return null;

        string json = File.ReadAllText(GameFile);

        return JsonSerializer.Deserialize<Game>(
            json,
            new JsonSerializerOptions
            {
                Converters = { new TwoDimensionalIntArrayConverter() }
            });
    }

    public static void SaveScores(List<Player> players)
    {
        string json = JsonSerializer.Serialize(players, new
        JsonSerializerOptions { WriteIndented = true });
        File.WriteAllText(ScoreFile, json);
    }

    public static List<Player> LoadScores()
    {
        if (!File.Exists(ScoreFile))
            return new List<Player>();

        string json = File.ReadAllText(ScoreFile);
        return JsonSerializer.Deserialize<List<Player>>(json) ?? new
        List<Player>();
    }
}
```

Листинг 8. TwoDimensionalIntArrayConverter.cs — сериализация массива int[,]

```
using System.Text.Json;
using System.Text.Json.Serialization;

namespace Gomoku;

public class TwoDimensionalIntArrayConverter : JsonConverter<int[,]>
{
    public override int[,]? Read(ref Utf8JsonReader reader, Type type,
        JsonSerializerOptions opt)
    {
        using var doc = JsonDocument.ParseValue(ref reader);

        int rows = doc.RootElement.GetArrayLength();
        int cols = doc.RootElement[0].GetArrayLength();
        int[,] result = new int[rows, cols];

        int r = 0;
        foreach (var row in doc.RootElement.EnumerateArray())
        {
            int c = 0;
            foreach (var v in row.EnumerateArray())
                result[r, c++] = v.GetInt32();
            r++;
        }

        return result;
    }

    public override void Write(Utf8JsonWriter w, int[,] value,
        JsonSerializerOptions opt)
    {
        w.WriteStartArray();
        for (int i = 0; i < value.GetLength(0); i++)
        {
            w.WriteStartArray();
            for (int j = 0; j < value.GetLength(1); j++)
                w.WriteNumberValue(value[i, j]);
            w.WriteEndArray();
        }
        w.WriteEndArray();
    }
}
```

Листинг 9. Gomoku.csproj — файл проекта

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net9.0</TargetFramework>
  </PropertyGroup>

</Project>
```

Ссылка на полный исходный код проекта

Полный проект доступен в открытом репозитории GitHub:

<https://github.com/taisiya18/Gomoku>